

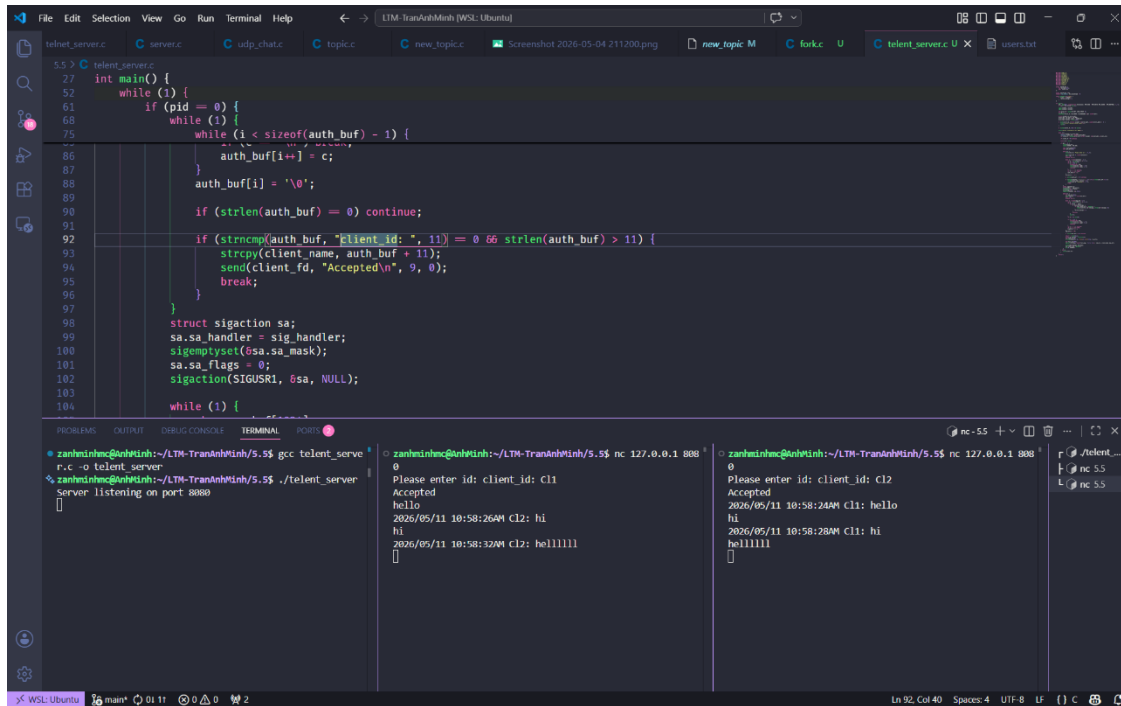
ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

BÁO CÁO BÀI TẬP
LẬP TRÌNH MẠNG

Trần Anh Minh - 20235382

HÀ NỘI, 04/2026

telnet server

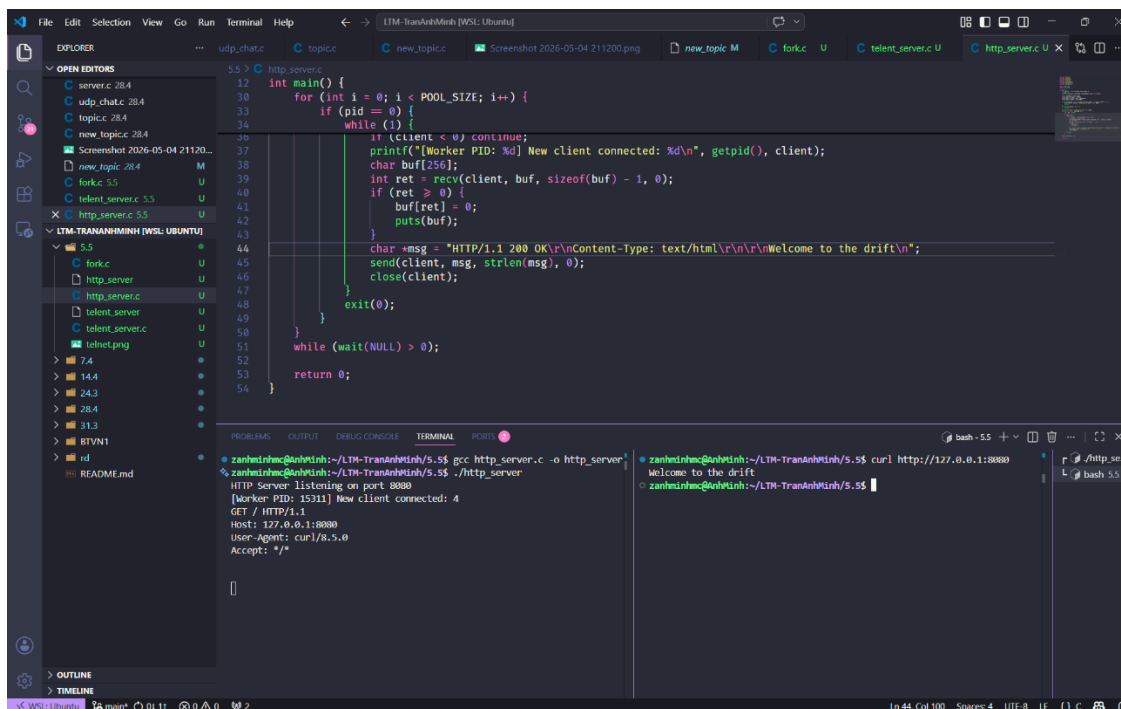


The screenshot shows a Visual Studio Code editor with a C program named `telnet_server.c`. The code implements a telnet server that listens on port 8080. It uses a `while` loop to accept connections. For each connection, it prompts the user to enter a client ID, reads the input, and then sends a response. The response includes the client ID and a timestamp. The code also includes a `sigaction` structure for signal handling.

```
127 int main() {
128     while (1) {
129         if (pid == 0) {
130             while (1) {
131                 while (1 < sizeof(auth_buf) - 1) {
132                     auth_buf[i++] = c;
133                 }
134                 auth_buf[i] = '\0';
135                 if (strlen(auth_buf) == 0) continue;
136                 if (strcmp(auth_buf, "client_id: ") == 0 && strlen(auth_buf) > 11) {
137                     strcpy(client_name, auth_buf + 11);
138                     send(client_fd, "Accepted\n", 9, 0);
139                     break;
140                 }
141             }
142             struct sigaction sa;
143             sa.sa_handler = sig_handler;
144             sigemptyset(&sa.sa_mask);
145             sa.sa_flags = 0;
146             sigaction(SIGUSR1, &sa, NULL);
147             while (1) {
```

The terminal output shows the server listening on port 8080. It receives two connections. The first connection sends "client_id: C11" and "hello". The second connection sends "client_id: C12" and "hi". The server responds with "Accepted" and a timestamp for each connection.

http server

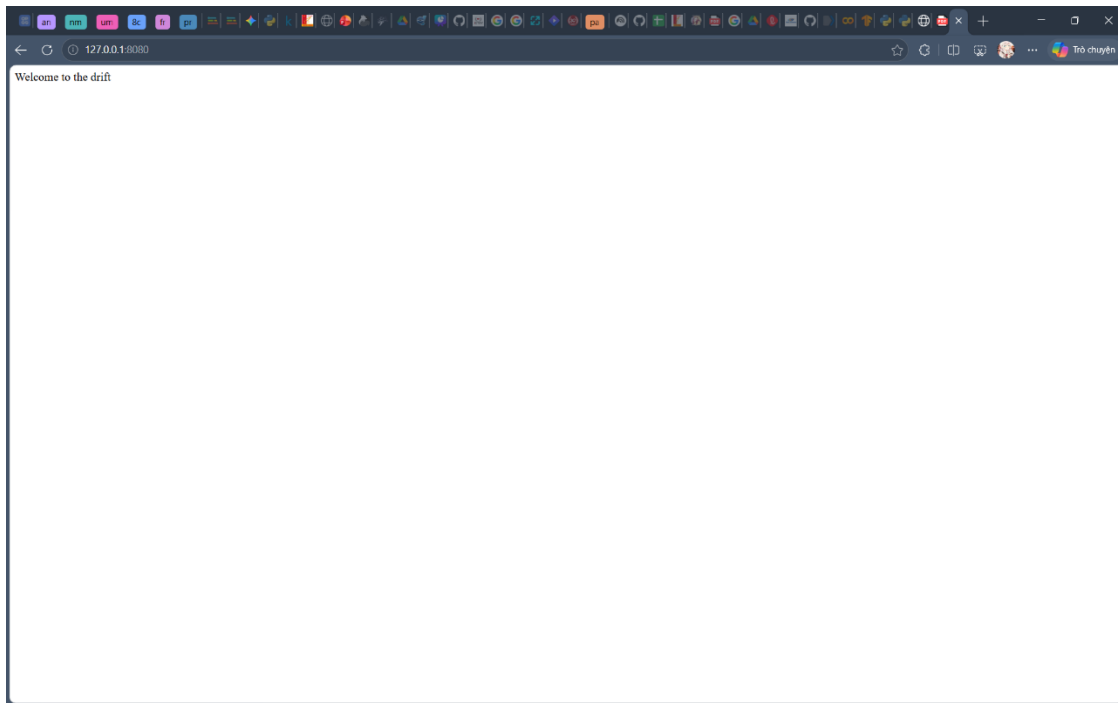


The screenshot shows a Visual Studio Code editor with a C program named `http_server.c`. The code implements an HTTP server that listens on port 8080. It uses a `while` loop to accept connections. For each connection, it prints a message, reads the request, and sends a response. The response is a simple "Welcome to the drift" message. The code also includes a `while` loop to wait for the next connection.

```
12 int main() {
13     for (int i = 0; i < POOL_SIZE; i++) {
14         if (pid == 0) {
15             while (1) {
16                 if (client < 0) continue;
17                 printf("[Worker PID: %d] New client connected: %d\n", getpid(), client);
18                 char buf[256];
19                 int ret = recv(client, buf, sizeof(buf) - 1, 0);
20                 if (ret >= 0) {
21                     buf[ret] = 0;
22                     puts(buf);
23                 }
24                 char *msg = "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\nWelcome to the drift\n";
25                 send(client, msg, strlen(msg), 0);
26                 close(client);
27             }
28             exit(0);
29         }
30     }
31     while (wait(NULL) > 0);
32     return 0;
33 }
```

The terminal output shows the server listening on port 8080. It receives a connection from "127.0.0.1:8080". The server responds with "Welcome to the drift".

Chạy bằng lệnh curl trong terminal



Mở bằng link trên trình duyệt

Time server

